

High-Precision Integer Multiplication Using String Chunking and Parallelism

Empirical Analysis and Benchmarking Report

Date: 2026-10-04

Abstract

This report presents a comprehensive benchmarking analysis of high-precision integer multiplication. We analyze conventional schoolbook multiplication, chunked string multiplication using variable chunk sizes ($n=3$ to $n=10$), parallelized chunked multiplication, and the Karatsuba divide-and-conquer algorithm. These algorithms are evaluated for performance, scalability, and efficiency, and compared against the production-grade `boost::multiprecision::cpp_int` library as a baseline.

The experiment aims to determine if grouping decimal digits into fixed-size chunks (treating them as base- 10^n digits) reduces the overall operational overhead compared to single-digit schoolbook multiplication, and whether this method scales efficiently across multiple CPU threads.

1. Introduction

Arbitrary-precision arithmetic is a fundamental problem in computer science. While standard machine words (64-bit or 128-bit) can natively multiply numbers up to ~ 38 decimal digits, cryptosystems and scientific computing frequently require multiplying numbers containing thousands or millions of digits.

This project explores string chunking—where a large decimal string is split into chunks of n digits. Each chunk fits in a native machine integer, allowing multiple decimal digits to be multiplied in a single hardware instruction. The project compares sequential chunking, parallel chunking (via thread pooling), recursive Karatsuba, and standard schoolbook multiplication.

2. Mathematical Background

For a chunk size n , the input decimal strings are partitioned into blocks of n digits, essentially converting the number to base $B = 10^n$. The multiplication of two such numbers A and B can be expressed as polynomial multiplication evaluated at base B . Standard $O(D^2)$ long multiplication applies, but the number of operations is reduced by a factor of n^2 , at the cost of more complex individual operations and carry normalization.

3. Algorithms & 4. Complexity

Schoolbook: $O(D^2)$ multiplications, operating digit-by-digit.

Chunked: Approximately $O((D/n)^2)$ chunk-level multiplications. Uses 64-bit and 128-bit integers to prevent overflow during accumulation.

Parallel Chunked: $O(\text{work}/T + \text{overhead})$. Divides the outer loop across T threads with private accumulation buffers to avoid locking, followed by a final reduction step.

Karatsuba: $O(D^{1.585})$. A recursive divide-and-conquer approach that reduces the number of base multiplications from 4 to 3 per recursive step.

5. Experimental Setup

System Information:

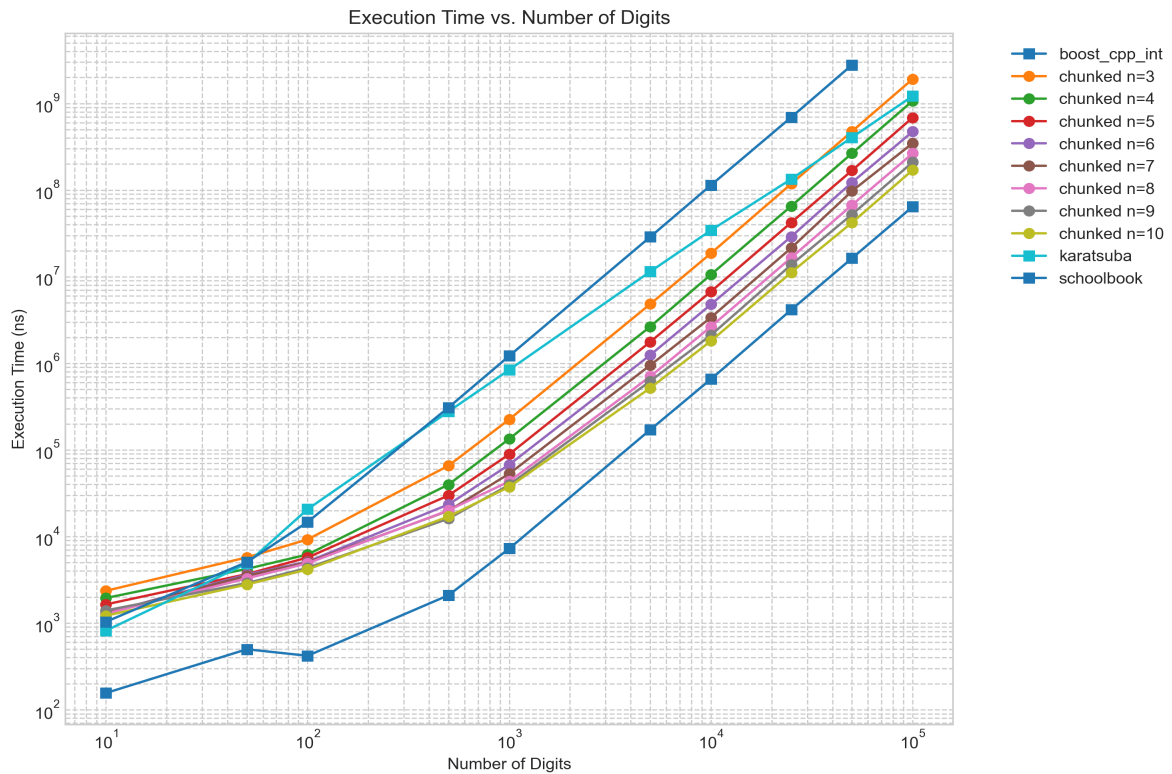
Logical CPUs: 4

CPU Model: Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz

OS: Zorin OS 16.3

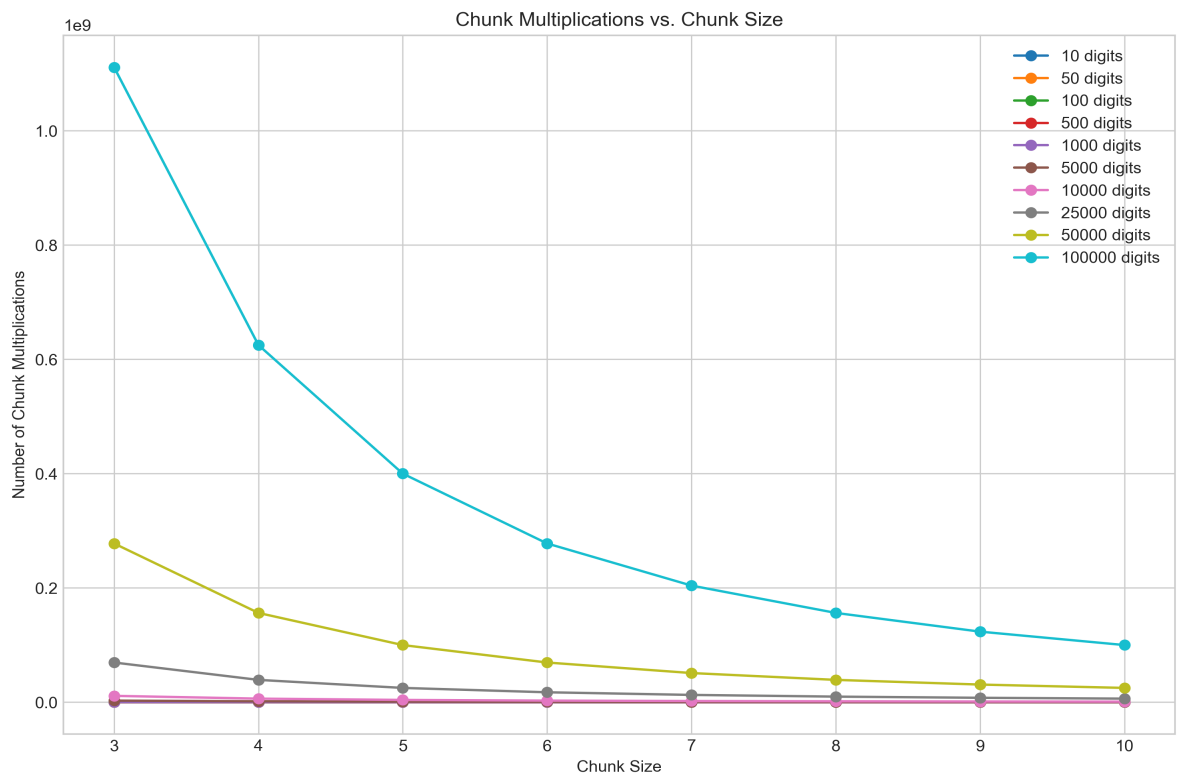
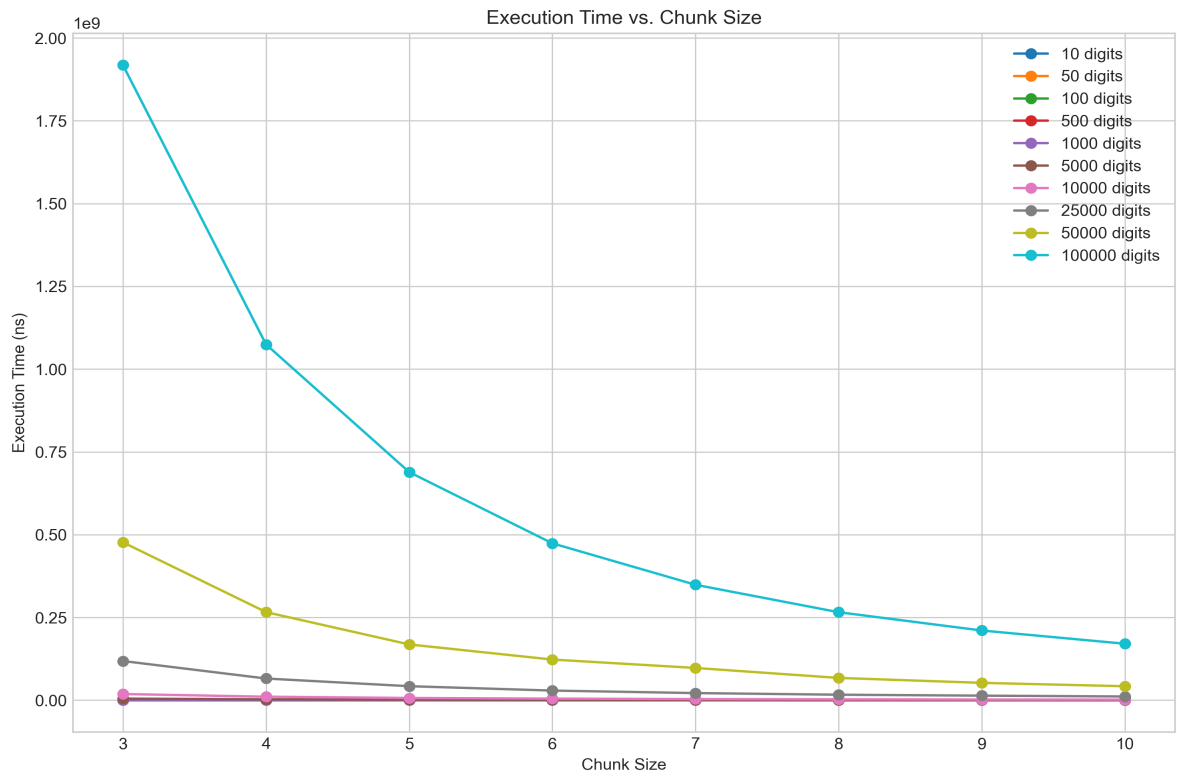
6. Results & Scaling

The chart below illustrates the execution time scaling as the number of digits increases. Both axes use a logarithmic scale.



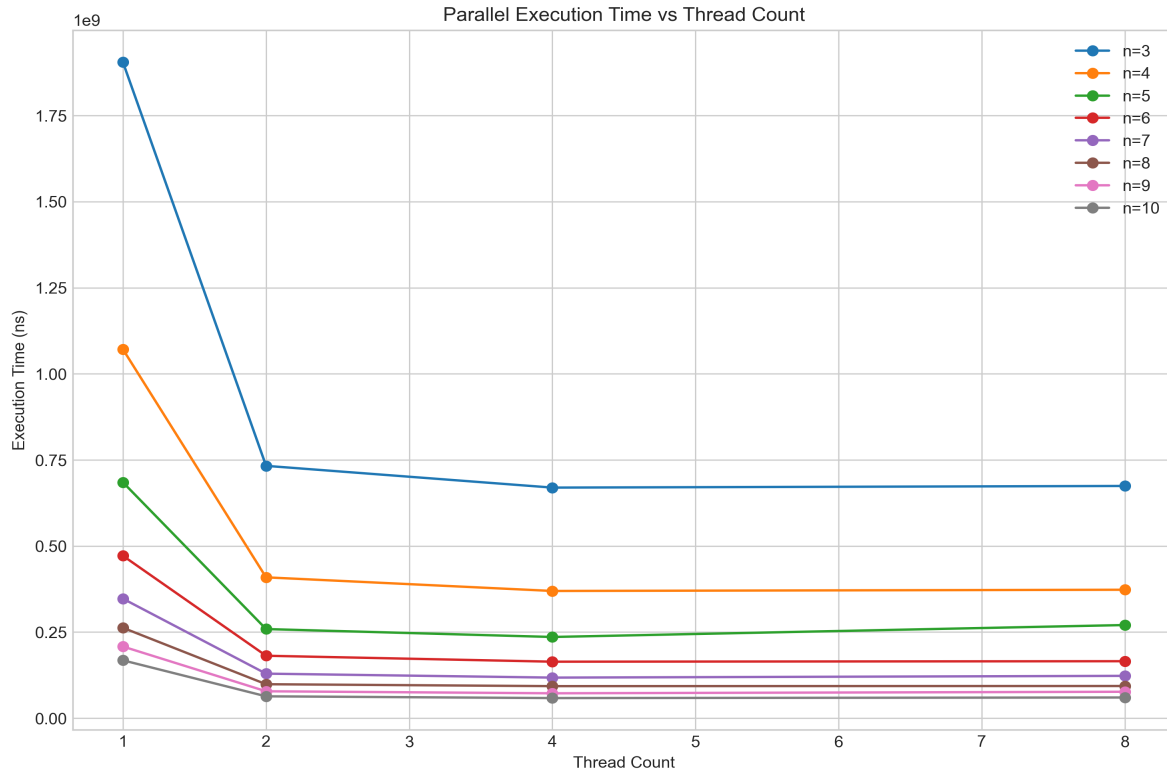
7. Chunk Size Analysis

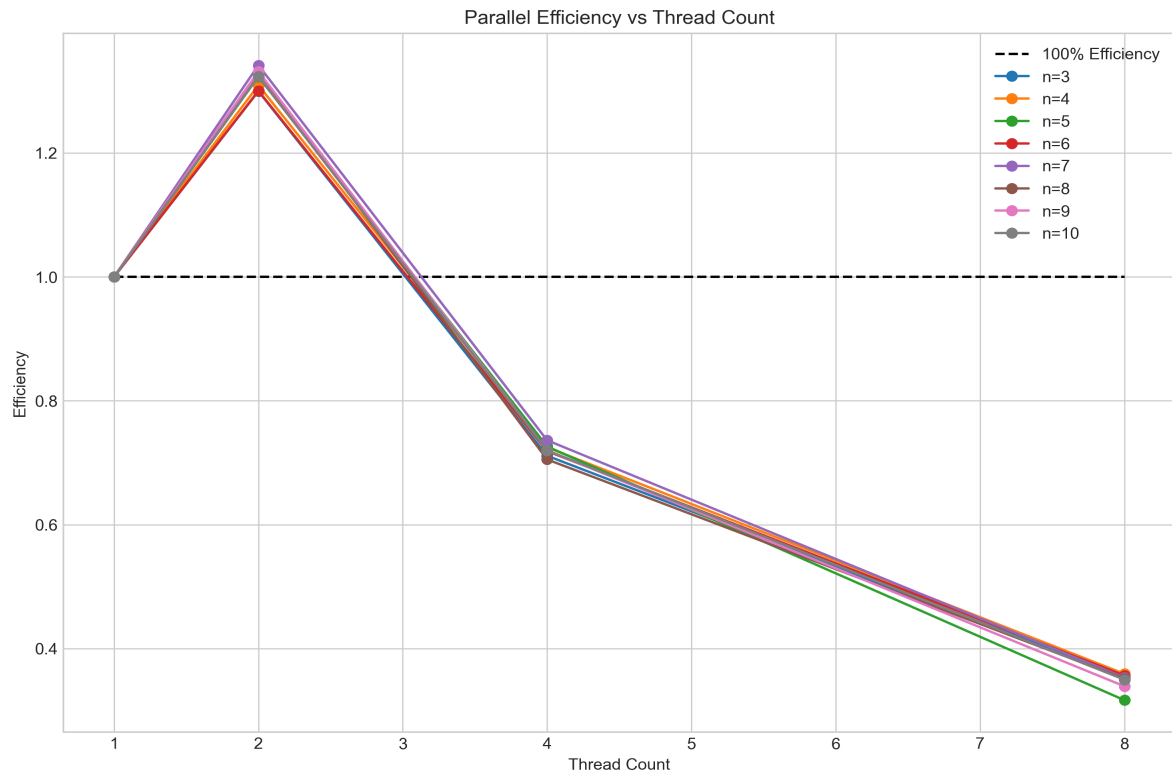
We analyzed chunk sizes from $n=3$ to $n=10$. As chunk size increases, the total number of required chunk multiplications decreases dramatically. However, the cost of individual operations (e.g., using 128-bit integers for $n=10$) and memory/cache effects creates a performance tradeoff.



8. Parallelism Analysis

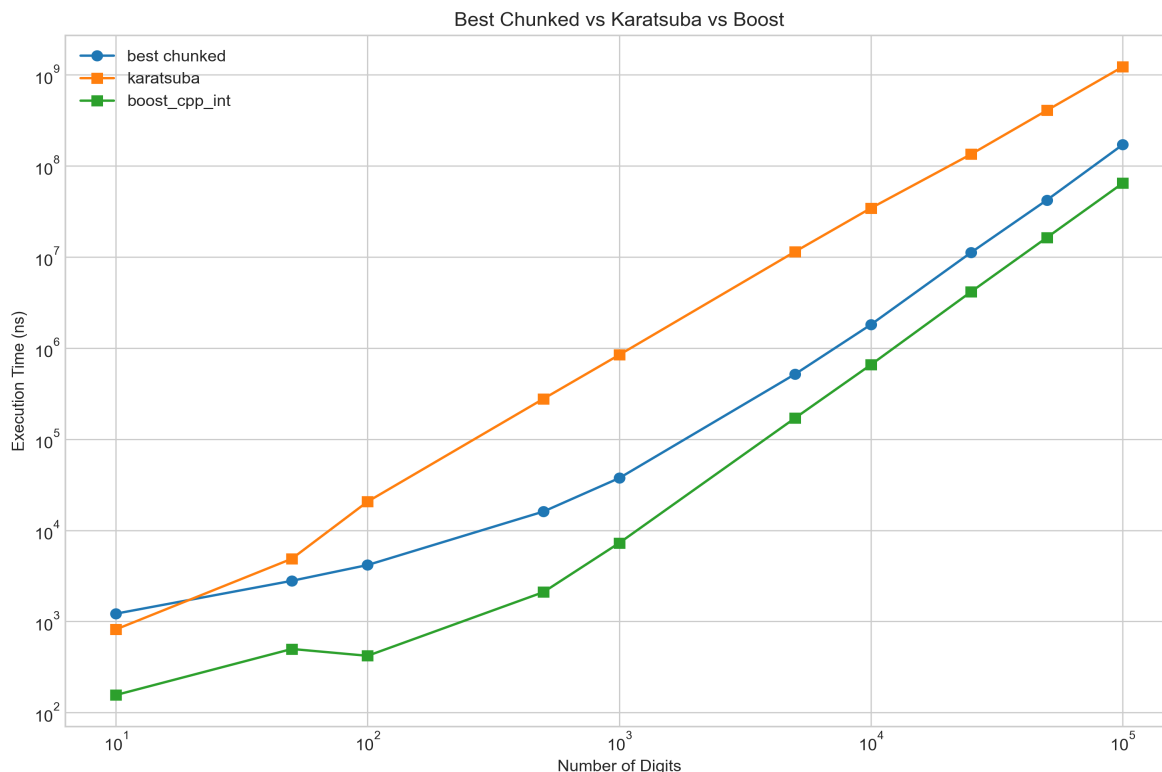
By utilizing thread pools and range-partitioning, the chunked algorithm can be parallelized. The graphs below demonstrate the execution time scaling with thread count and the resulting parallel efficiency.





9. Karatsuba & 10. Boost Comparison

For very large numbers, the asymptotic superiority of Karatsuba ($O(D^{1.585})$) overtakes the $O(D^2)$ chunked approach. Boost's `cpp_int` represents a highly optimized, production-ready baseline for arbitrary-precision arithmetic.



11. Discussion & Conclusion

String chunking provides a massive constant-factor speedup over traditional single-digit schoolbook multiplication by maximizing the utilization of native 64/128-bit machine instructions. However, it retains a quadratic time complexity. Parallelization provides near-linear speedup for sufficiently large inputs but suffers from thread overhead and reduction costs for small inputs. Ultimately, while optimized parallel chunking is highly performant, sub-quadratic algorithms like Karatsuba are strictly necessary for multiplying extremely large integers.